MEMORIA DE ACTIVIDADES

Actividad 2.1 — Problemas de búsqueda
Actividad 2.2 — Espacios de búsqueda

Modelado de dos problemas reales como problemas de búsqueda, representación de sus espacios de estados como grafos dirigidos y ponderados, y programación de un generador de estados y de los grafos con NetworkX en Python

Asignatura: Fundamentos de Inteligencia Artificial

Estudiante: Julissa Marin Flor

Docente: Javier García Lara

Institución: Universidad Politécnica de Atlautla

Fecha: 18 de octubre de 2026

1. Introducción

Un programa de inteligencia artificial casi siempre puede describirse como un agente que busca: desde una ambulancia que debe llegar al hospital por el camino más rápido hasta un dron que debe repartir en varios puntos con la menor duración posible. Formalizar esos casos como *problemas de búsqueda* permite aplicar algoritmos conocidos y compararlos de forma objetiva.

Esta memoria corresponde a las Actividades 2.1 (Problemas de búsqueda) y 2.2 (Espacios de búsqueda) del curso. Los objetivos cumplidos son:

- Modelar dos problemas reales como problemas de búsqueda, identificando sus componentes: estados, acciones, función de transición, coste y meta.
- Representar cada espacio de estados como un grafo dirigido y ponderado.
- Programar en Python un generador de estados y construir los grafos con la biblioteca NetworkX.
- Ejecutar algoritmos de búsqueda clásicos (BFS, Dijkstra y A*) y comparar sus resultados, verificando la optimalidad de la solución del problema 2 por enumeración exhaustiva.

Todo el código, las figuras y la salida de ejecución forman parte del repositorio de esta asignatura y se pueden regenerar con un solo comando (sección 7).

2. Marco teórico

Un problema de búsqueda se define como la tupa P = <S, s0, A, coste, meta>:

Componente	**Significado**
S (estados)	conjunto de configuraciones posibles del sistema
s0 (inicial	estado de partida del agente
A(s) (acciones)	acciones aplicables en el estado s
Transición	función que indica a qué estado lleva cada acción
Coste	suma de costes de las transiciones (no negativos)
Meta	estado (o conjunto de estados) objetivo

El espacio de estados es el conjunto S junto con las transiciones entre ellos. Cuando cada estado puede ir a otro de forma direccional (no simétrica) el grafo es **dirigido** (digrafo, `nx.DiGraph`); cuando cada transición tiene un coste distinto el grafo es **ponderado**, y ese coste se guarda en el atributo `weight` de la arista. NetworkX exige ese nombre de atributo para sus algoritmos de camino ponderado.

Algoritmos empleados (Russell & Norvig, 2021):

- **Búsqueda en amplitud (BFS)**: explora por niveles; no mira los costes y garantiza el camino con menos aristas (acciones), no el más barato.
- **Dijkstra**: expande siempre el estado de menor coste acumulado; es óptimo en grafos con pesos no negativos.
- **A***: como Dijkstra pero ordena por f(n) = g(n) + h(n). Si la heurística h es admisible (nunca sobreestima el coste real) la solución sigue siendo óptima y suelen expandirse menos estados.

3. Problema 1 — Ruteo de una ambulancia en red urbana

3.1 Descripción del problema real

Una ambulancia parte del cuartel y debe llegar al Hospital General. La red urbana tiene calles de sentido único (por eso el grafo es dirigido) y el tiempo de traslado de cada calle cambia con el tráfico de la hora pico (por eso el grafo es ponderado, con minutos como coste). El equipo de coordinate quiere saber qué ruta minimiza el tiempo de atención.

3.2 Modelado como problema de búsqueda

Componente	Definición en este problema
Estados (S)	las 12 intersecciones de la red: A … L (p. ej. A = Cuartel, L = Hospital)
Estado inicial (s0)	'A' = Cuartel
Acciones	circular por una calle adyacente respetando el sentido único; una acción por cada arista saliente
Transición	la acción desplaza la ambulancia a la intersección destino
Coste	minutos de traslado por esa calle en hora pico (peso de la arista)
Meta	'L' = Hospital General
Espacio de estados	grafo dirigido y ponderado de 12 nodos y 18 aristas; aquí el grafo físico y el de estados coinciden

3.3 Representación del espacio de estados como grafo

La exploración en amplitud desde el estado inicial genera 12 estados y 18 transiciones en 6 niveles. NetworkX lo representa como `nx.DiGraph` con el atributo `weight` en cada arista.

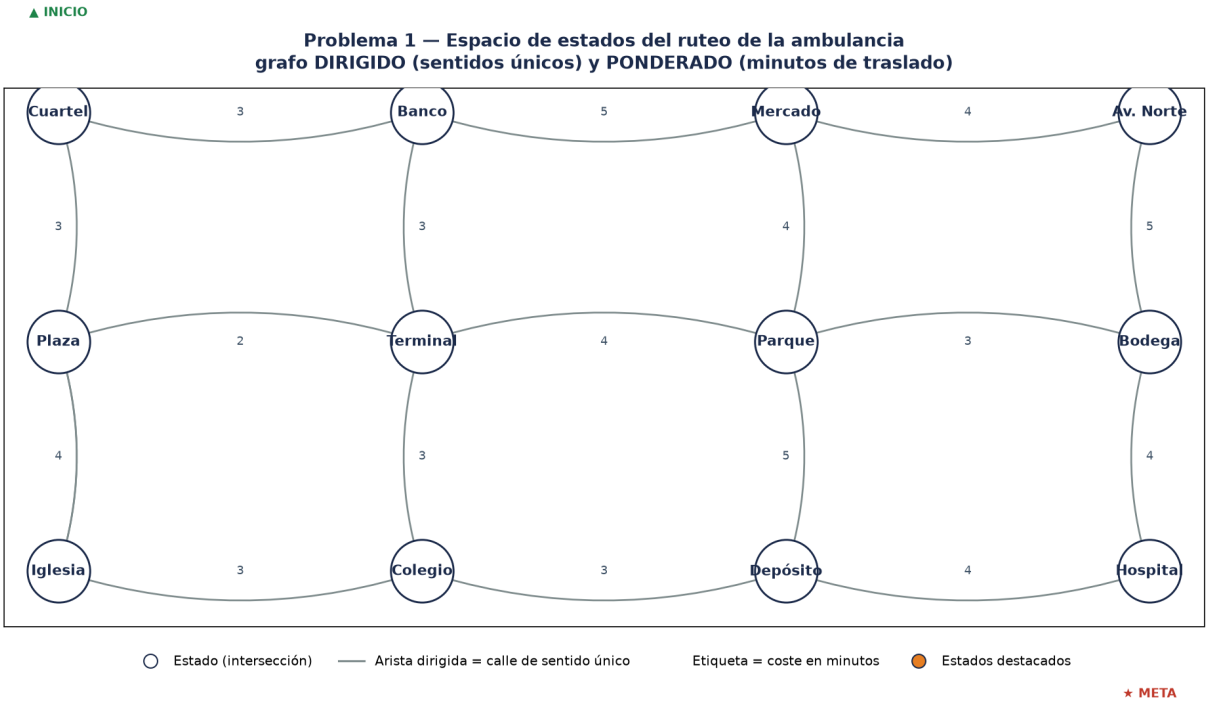


Figura 1. Espacio de estados del problema 1: grafo dirigido (sentidos únicos) y ponderado (minutos). Naranja: estado inicial y meta.

Tabla de aristas (calles) con su coste:

Arista	Origen	Destino	Calle (acción)	Coste (min)
A → B	Cuartel	Banco	Av. Central	3
B → C	Banco	Mercado	Av. Central	5
C → D	Mercado	Av. Norte	Av. Central	4
H → G	Bodega	Parque	Calle Morelos	3
G → F	Parque	Terminal	Calle Morelos	4
F → E	Terminal	Plaza	Calle Morelos	2
I → J	Iglesia	Colegio	Calle 5 de Mayo	3
J → K	Colegio	Depósito	Calle 5 de Mayo	3
K → L	Depósito	Hospital	Calle 5 de Mayo	4
I → E	Iglesia	Plaza	Calle Allende	4
E → A	Plaza	Cuartel	Calle Allende	3
E → I	Plaza	Iglesia	Calle Allende	5
B → F	Banco	Terminal	Av. Hidalgo	3
F → J	Terminal	Colegio	Av. Hidalgo	3
K → G	Depósito	Parque	Calle Zaragoza	5

G → C	Parque	Mercado	Calle Zaragoza	4
D → H	Av. Norte	Bodega	Av. Madero	5
H → L	Bodega	Hospital	Av. Madero	4

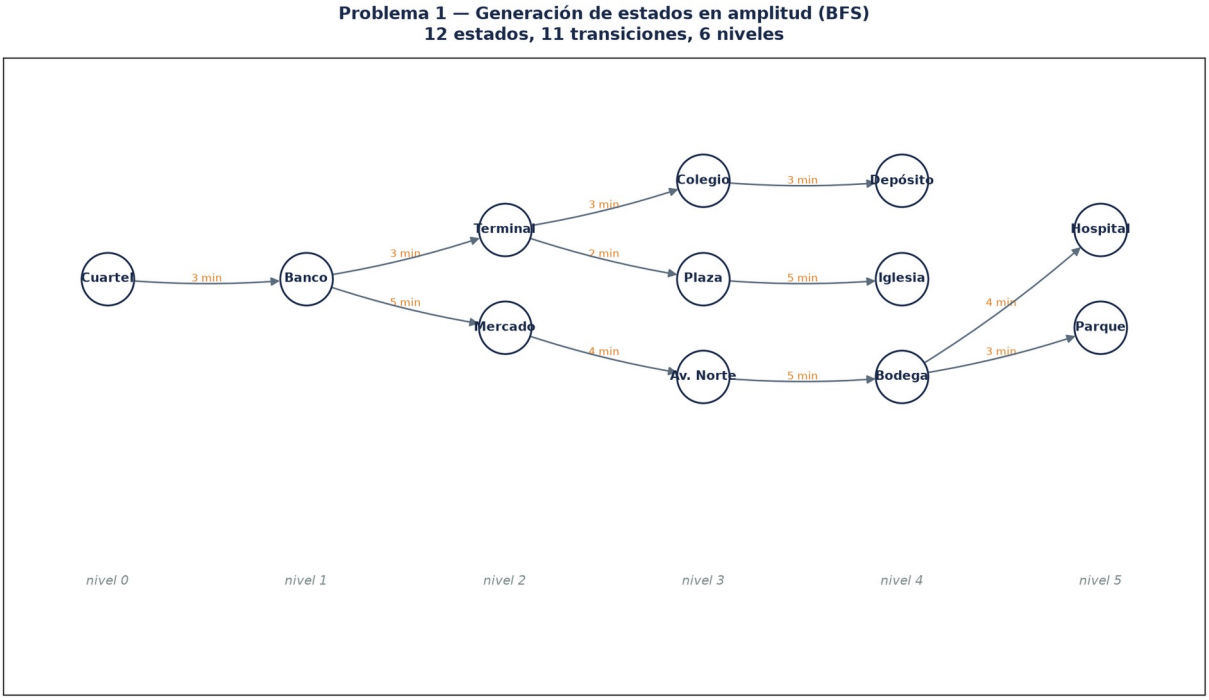


Figura 2. Generación de estados en amplitud: el árbol muestra de qué estado se descubrió cada uno y el coste de cada acción.

3.4 Generador de estados (Python)

La clase `GeneradorRuteoAmbulancia` implementa la interfaz del problema (estado_inicial, es_meta, acciones, sucesores) y la función `explorar_espacio` recorre el generador en amplitud para obtener todos los estados alcanzables y sus transiciones. Listado real del archivo `codigo/generador_estados.py`:

```
  def sucesores(self, estado: Estado) -> List[Transicion]:
    """Todos los estados alcanzables desde `estado` con su coste."""
    return [
      Transicion(
        accion=f"Tomar {nombre} hacia {destino}",
        origen=estado,
        destino=destino,
        coste=minutos,
      )
      for (origen, destino), (minutos, nombre) in self.red.items()
      if origen == estado
    ]
```

Exploración del espacio de estados (generador → conjunto de estados):

```
def explorar_espacio(generador, limite: int = 100_000) -> EspacioEstados:
    """
    Genera el espacio de estados mediante exploración en amplitud (BFS)
    desde el estado inicial, hasta agotar los estados alcanzables (o
    alcanzar el límite de seguridad `limite`).

    Devuelve los estados descubiertos, su profundidad y las transiciones
    (accion, coste) entre ellos: es la materia prima para dibujar el grafo
    del espacio de estados con NetworkX.
    """
    espacio = EspacioEstados()
    inicial = generador.estado_inicial
    cola = deque([inicial])
    vistos = {inicial}
    espacio.nivel[inicial] = 0
    espacio.transiciones[inicial] = []

    while cola:
        estado = cola.popleft()
        espacio.estados.append(estado)
        sucesores = generador.sucesores(estado)
        espacio.transiciones[estado] = sucesores
        for tr in sucesores:
            if tr.destino not in vistos:
                vistos.add(tr.destino)
                espacio.nivel[tr.destino] = espacio.nivel[estado] + 1
                espacio.transiciones[tr.destino] = []
                if len(vistos) > limite:
                    raise RuntimeError(
                        f"El espacio supera el límite de {limite} estados; "
                        "ajusta el problema o aumenta el límite."
                    )
                cola.append(tr.destino)

    espacio.alcanzables = len(vistos)
    return espacio
```

3.5 Construcción del grafo con NetworkX

El archivo `codigo/construir_grafos.py` convierte los datos y el espacio generado en grafos dirigidos y ponderados, y dibuja las figuras:

```
def grafo_red_calles() -> nx.DiGraph:
```

```
    """Grafo del problema 1: intersecciones (nodos) y calles (aristas dirigidas)."""
    g = nx.DiGraph()
    for nodo, nombre in NOMBRES_RED.items():
        g.add_node(nodo, nombre=nombre, pos=POSICIONES_RED[nodo])
    for (o, d), (minutos, calle) in RED_CALLES.items():
        g.add_edge(o, d, weight=minutos, calle=calle, label=f"{minutos:g}")
    return g


def grafo_espacio_estados(espacio: EspacioEstados) -> nx.DiGraph:
    """
    Vuelca la exploración del generador de estados a un grafo dirigido:
    nodos = estados alcanzables, aristas = acciones con su coste (`weight`).
    El atributo `subset` (nivel de profundidad) permite dibujarlo por capas.
    """
    g = nx.DiGraph()
    for estado in espacio.estados:
        g.add_node(estado, subset=espacio.nivel[estado])
    for estado, transiciones in espacio.transiciones.items():
        for tr in transiciones:
            g.add_edge(tr.origen, tr.destino, weight=tr.coste,
                       accion=tr.accion, label=f"{tr.coste:g}")
    return g
```

3.6 Resultados de búsqueda

Algoritmo	Coste	Calles (aristas)	Estados expandidos	Ruta encontrada
BFS (amplitud)	21 min	5	12	A → B → C → D → H → L
Dijkstra	16 min	5	10	A → B → F → J → K → L
A*	16 min	5	9	A → B → F → J → K → L

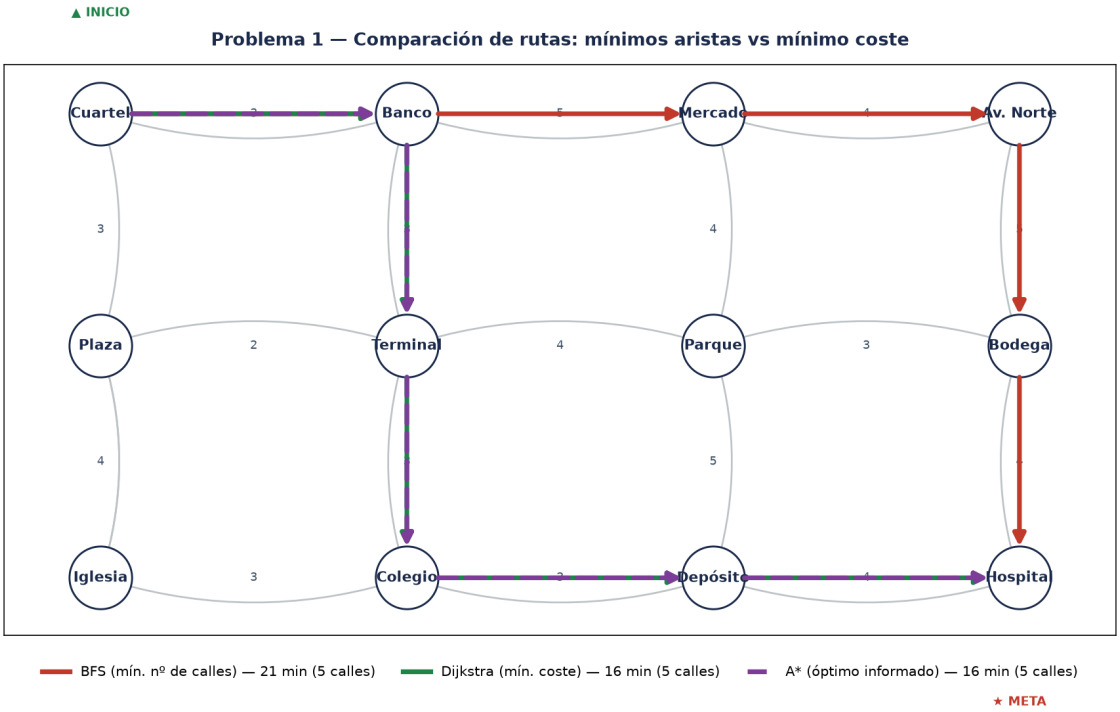


Figura 3. Comparación de las rutas: BFS elige la avenida congestionada; Dijkstra y A encuentran la ruta de menor tiempo.*

Análisis: BFS devuelve 21 min, aunque con el mismo número de calles que el óptimo (5), porque **no mira los pesos**: solo cuenta aristas. Dijkstra y A* encuentran 16 min, 5 min menos. La heurística usada en A* es h(n) = (menor coste por unidad de distancia) × distancia(n, meta), que es admisible porque se calcula con el coste mínimo por unidad de todas las aristas; por eso A* conserva la optimalidad y expandió sólo 9 estados frente a 10 de Dijkstra.

4. Problema 2 — Ruta de un dron de reparto con varias entregas

4.1 Descripción del problema real

Un dron de reparto debe salir de su base, entregar en cuatro puntos (farmacia, clínica, mercado y escuela) y regresar. Los tiempos de vuelo dependen del corredor aéreo: hay desvíos por zonas de exclusión y viento, de modo que volar de A a B no cuesta lo mismo que de B a A (tiempos asimétricos → grafo dirigido). El dron no puede volver a la base antes de terminar las entregas (restricción de autonomía).

4.2 Modelado como problema de búsqueda

Componente	Definición en este problema
Estados (S)	(nodo_actual, conjunto_de_entregas_realizadas); p. ej. (P1, {P1, P2}) = en la farmacia, ya entregó a P1 y P2
Estado inicial (s0)	(BASE, ∅): el dron en la base sin entregar
Acciones	volar a un punto de entrega aún no visitado; cuando el conjunto está

	completo, regresar a la BASE
Transición	añade el punto al conjunto y mueve el dron al nuevo nodo
Coste	minutos de vuelo del tramo (peso de la arista)
Meta	(BASE, {P1, P2, P3, P4})
Espacio de estados	grafo dirigido y ponderado generado explícitamente: 34 estados y 56 transiciones

A diferencia del problema 1, aquí el grafo físico (7 nodos) y el espacio de estados (34 estados) NO coinciden: cada estado recuerda qué entregas ya se hicieron, por eso se dice que el espacio es *explícito* y se construye con el generador.

4.3 Red física y espacio de estados

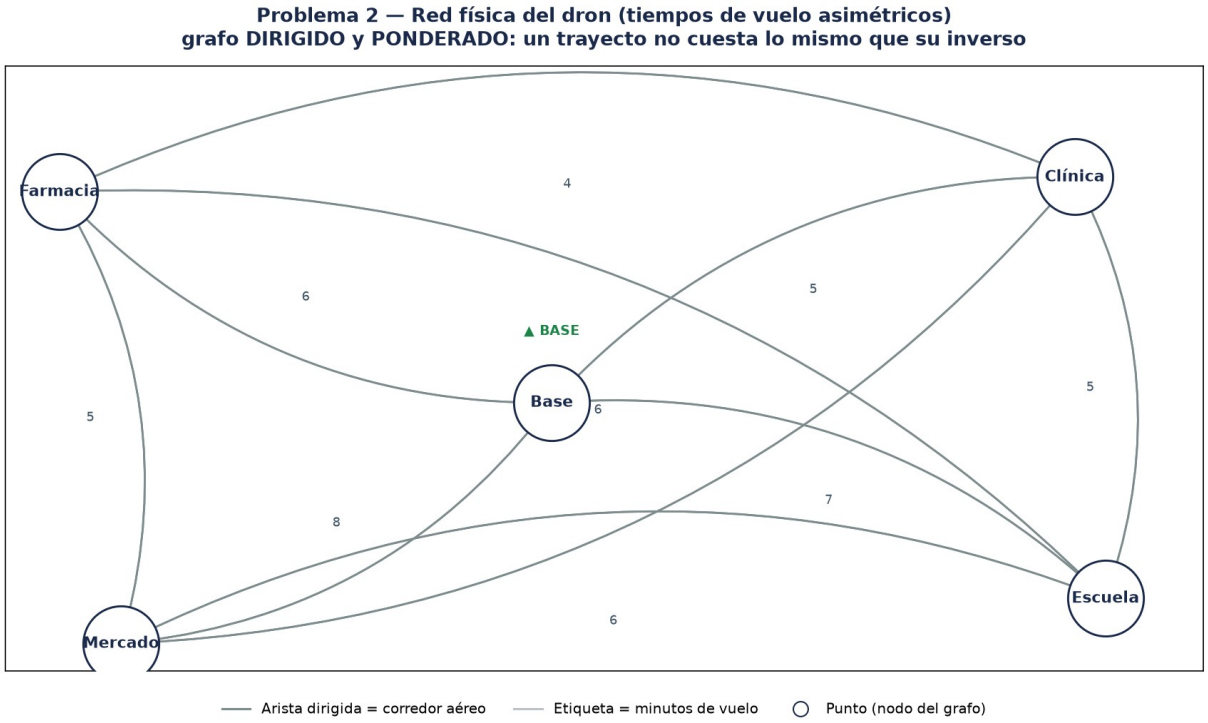


Figura 4. Red física del dron: grafo dirigido y ponderado con tiempos asimétricos (p. ej. P2→P3 = 8 min frente a P3→P2 = 6 min).

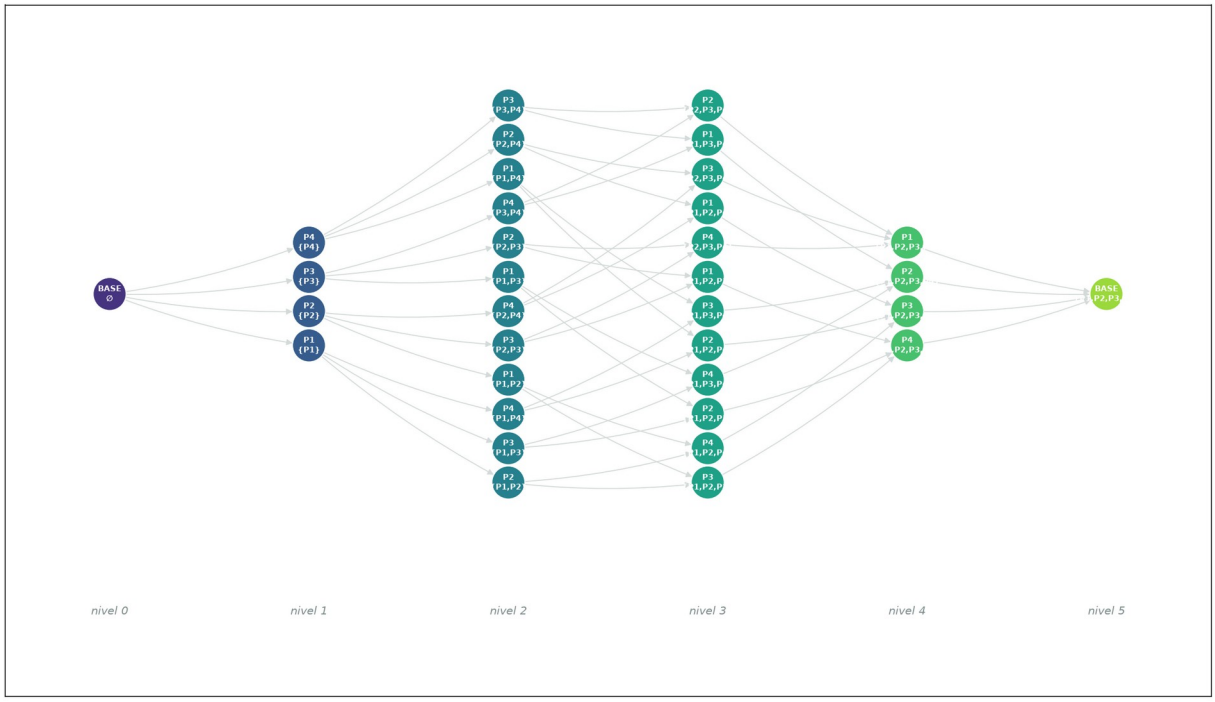


Figura 5. Espacio de estados generado: 34 estados (capas = número de acciones realizadas) y 56 transiciones.

4.4 Generador de estados (Python)

La clase `GeneradorRutaDron` aplica las acciones sobre estados con memoria (conjunto de entregas); `explorar_espacio` es la misma función del problema 1, reutilizada sin cambios:

```
def sucesores(self, estado: Estado) -> List[Transicion]:
    nodo, hechas = estado  # type: ignore[misc]
    # Regreso a la base sólo cuando ya se completaron las entregas.
    if hechas == self.total:
        if nodo == self.base:
            return []
        return [Transicion(
            accion=f"Regresar a {self.base}",
            origen=estado,
            destino=(self.base, self.total),
            coste=self.tiempos[(nodo, self.base)],
        )]
    # Elegir el siguiente punto de entrega aún no visitado.
    return [
        Transicion(
            accion=f"Volar a {self.puntos[p]} ({p})",
            origen=estado,
            destino=(p, hechas | {p}),
            coste=self.tiempos[(nodo, p)],
        )
        for p in sorted(self.puntos)
        if p not in hechas
    ]
```

4.5 Resultados de búsqueda y verificación

Algoritmo	Coste	Orden de entrega	Expandidos
BFS (amplitud)	26 min	BASE → P1 → P2 → P3 → P4 → BASE	34
Dijkstra	22 min	BASE → P3 → P4 → P1 → P2 → BASE	34
Fuerza bruta (24 órdenes)	22 min	BASE → P3 → P4 → P1 → P2 → BASE	—

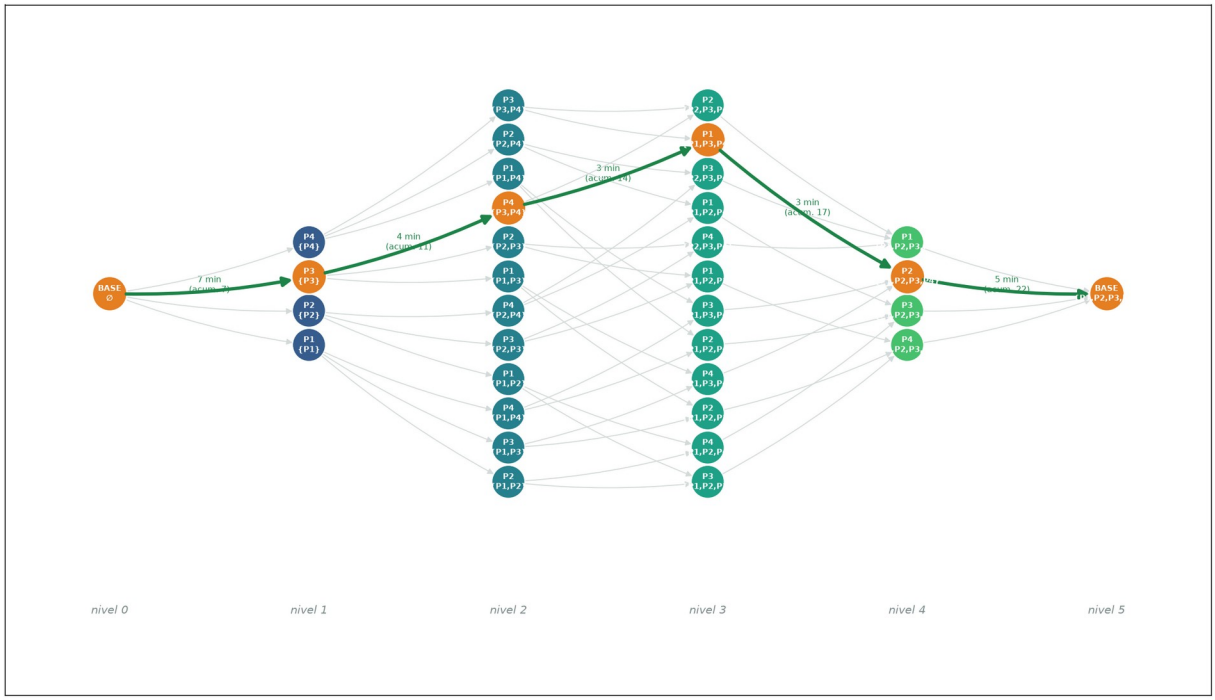


Figura 6. Solución óptima en el espacio de estados: estados de la ruta en naranja y coste acumulado en cada transición.

Verificación: la enumeración exhaustiva de los 4! = 24 ordenes posibles devuelve 22 min, exactamente el coste de Dijkstra (22 min) → la solución es óptima. BFS, que ignora los pesos, entrega 26 min, 4 min peor.

5. Análisis comparativo de los dos problemas

Criterio	Problema 1 (ambulancia)	Problema 2 (dron)
Estados	12 intersecciones	34 pares (nodo, entregas)
Acciones	circular por una calle permitida	volar a un punto sin entregar / regresar
Coste	minutos de traslado	minutos de vuelo
Meta	llegar al hospital	volver a la base con todo entregado
¿Dirigido?	sí (sentidos únicos)	sí (tiempos asimétricos)
¿Ponderado?	sí (minutos por calle)	sí (minutos por tramo)
Grafo físico vs estados	coinciden (12 = 12)	no coinciden (7 vs 34)

Espacio generado por	la propia red	el generador de estados (BFS)
Mejor algoritmo	A* (óptimo con menos expansiones)	Dijkstra (óptimo verificado)
Solución	16 min	22 min

Los dos casos ilustran por qué importa el modelo: mismo marco formal <S, s0, A, coste, meta> y mismos algoritmos, pero un espacio de estados pequeño y directo en el primero y un espacio con memoria (conjunto de entregas) en el segundo, que obliga a generar el grafo explícitamente.

6. Conclusiones

1. Los dos problemas reales quedaron modelados con todos los componentes de un problema de búsqueda y representados como grafos dirigidos y ponderados (12/18 y 34/56 nodos/aristas).
2. El generador de estados es único y reutilizable: basta con implementar la interfaz (estado_inicial, es_meta, acciones, sucesores) para que la misma exploración en amplitud construya el espacio de estados de cualquier problema.
3. Minimizar el número de acciones (BFS) no garantiza minimizar el coste: en el problema 1 BFS da 21 min frente a 16 min de Dijkstra y A*, y en el problema 2 da 26 min frente a 22 min.
4. A* con una heurística admisible logra la misma solución óptima que Dijkstra expandiendo menos estados, lo que lo hace atractivo para grafos grandes.
5. La optimalidad de la solución del problema 2 se verificó de forma independiente con enumeración exhaustiva de los 24 ordenes posibles.
6. Extensión natural: sustituir el coste uniforme por tráfico en tiempo real (aristas con pesos dinámicos) o usar búsqueda A* con una heurística de vuelo directo para el dron.

7. Ejecución del código

Requisitos: Python 3.10 o superior y las dependencias de `codigo/requirements.txt` (NetworkX y Matplotlib).

```
cd actividad_2.1_2.2_problemas_busqueda/codigo
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python main.py
```

La ejecución genera las seis figuras de `figuras/` y el informe `resultados_ejecucion.txt` con la misma información que se recoge en esta memoria. Los módulos `generador_estados.py`, `construir_grafos.py` y `busqueda.py` también se pueden ejecutar por separado.

8. Referencias

[1] RUSSELL, S. y NORVIG, P. (2021). Artificial Intelligence: A Modern Approach, 4.ª ed. Pearson. Capítulos 3–4 (búsqueda de grafos).

[2] NILSSON, N. J. (1998). Artificial Intelligence: A New Synthesis. Morgan Kaufmann. (Espacios de estados y algoritmos de búsqueda.)

[3] HART, P. E., NILSSON, N. J. y RAPHAEL, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107. DOI: 10.1109/TSSC.1968.300136.

[4] NETWORKX DEVELOPERS (s. f.). Documentation — Graphs and Networks. Disponible en: https://networkx.org/documentation/stable/reference/index.html

[5] PYTHON SOFTWARE FOUNDATION (s. f.). Python Documentation. Disponible en: https://docs.python.org/3/

[6] OPENCODE (2026). OpenCode: agente de inteligencia artificial para programación y análisis [software]. Disponible en: https://opencode.ai.

Anexo A — Listado del código fuente (extractos)

Los extractos se obtuvieron automáticamente del código ejecutado con inspect.getsource(); el código completo está en la carpeta `codigo/` del repositorio.

A.1 generador_estados.py — exploración del espacio de estados

```
def explorar_espacio(generador, limite: int = 100_000) -> EspacioEstados:
    """
    Genera el espacio de estados mediante exploración en amplitud (BFS)
    desde el estado inicial, hasta agotar los estados alcanzables (o
    alcanzar el límite de seguridad `limite`).

    Devuelve los estados descubiertos, su profundidad y las transiciones
    (accion, coste) entre ellos: es la materia prima para dibujar el grafo
    del espacio de estados con NetworkX.
    """
    espacio = EspacioEstados()
    inicial = generador.estado_inicial
    cola = deque([inicial])
    vistos = {inicial}
    espacio.nivel[inicial] = 0
    espacio.transiciones[inicial] = []

    while cola:
        estado = cola.popleft()
        espacio.estados.append(estado)
        sucesores = generador.sucesores(estado)
        espacio.transiciones[estado] = sucesores
        for tr in sucesores:
            if tr.destino not in vistos:
                vistos.add(tr.destino)
                espacio.nivel[tr.destino] = espacio.nivel[estado] + 1
                espacio.transiciones[tr.destino] = []
                if len(vistos) > limite:
                    raise RuntimeError(
                        f"El espacio supera el límite de {limite} estados; "
                        "ajusta el problema o aumenta el límite."
                    )
                cola.append(tr.destino)

    espacio.alcanzables = len(vistos)
    return espacio
```

A.2 busqueda.py — búsqueda en amplitud y Dijkstra

```
def busqueda_amplitud(g: nx.DiGraph, inicio: Estado, meta: Estado) -> Resultado:
    """BFS: mínimo número de aristas entre `inicio` y `meta` (ignora pesos)."""
    padre: Dict[Estado, Optional[Estado]] = {inicio: None}
    cola = [inicio]
    cabeza = 0
    expandidos = 0
    while cabeza < len(cola):
        estado = cola[cabeza]
        cabeza += 1
        expandidos += 1
```

```
        if estado == meta:
            break
        for vecino in g.successors(estado):
            if vecino not in padre:
                padre[vecino] = estado
                cola.append(vecino)
    if meta not in padre:
        return Resultado("BFS (amplitud)", [], float("inf"), expandidos, len(padre))
    camino, actual = [], meta
    while actual is not None:
        camino.append(actual)
        actual = padre[actual]
    camino.reverse()
    coste = sum(g[o][d]["weight"] for o, d in zip(camino, camino[1:]))
    return Resultado("BFS (amplitud)", camino, coste, expandidos, len(padre))


def dijkstra(g: nx.DiGraph, inicio: Estado, meta: Estado) -> Resultado:
    """Dijkstra: camino de coste total mínimo (óptimo) en grafos con pesos ≥ 0."""
    mejor: Dict[Estado, float] = {inicio: 0.0}
    padre: Dict[Estado, Optional[Estado]] = {inicio: None}
    cola: List[Tuple[float, int, Estado]] = [(0.0, 0, inicio)]
    contador = 0
    expandidos = 0
    vistos = set()

    while cola:
        dist, _, estado = heapq.heappop(cola)
        if estado in vistos:
            continue
        vistos.add(estado)
        expandidos += 1
        if estado == meta:
            break
        for vecino, data in g[estado].items():
            nuevo = dist + data["weight"]
            if vecino not in mejor or nuevo < mejor[vecino]:
                mejor[vecino] = nuevo
                padre[vecino] = estado
                contador += 1
                heapq.heappush(cola, (nuevo, contador, vecino))

    if meta not in mejor:
        return Resultado("Dijkstra", [], float("inf"), expandidos, len(padre))
    camino, actual = [], meta
    while actual is not None:
        camino.append(actual)
        actual = padre[actual]
    camino.reverse()
    return Resultado("Dijkstra", camino, mejor[meta], expandidos, len(padre))
```

A.3 busqueda.py — A* y heurística admisible

```
def a_estrella(g: nx.DiGraph, inicio: Estado, meta: Estado,
               h: Callable[[Estado], float]) -> Resultado:
```

```
    """A* con heurística admisible h: óptimo y, en general, expande menos."""
    g_score: Dict[Estado, float] = {inicio: 0.0}
    padre: Dict[Estado, Optional[Estado]] = {inicio: None}
    cola: List[Tuple[float, int, Estado]] = [(h(inicio), 0, inicio)]
    contador = 0
    expandidos = 0
    vistos = set()

    while cola:
        f, _, estado = heapq.heappop(cola)
        if estado in vistos:
            continue
        vistos.add(estado)
        expandidos += 1
        if estado == meta:
            break
        for vecino, data in g[estado].items():
            nuevo = g_score[estado] + data["weight"]
            if vecino not in g_score or nuevo < g_score[vecino]:
                g_score[vecino] = nuevo
                padre[vecino] = estado
                contador += 1
                heapq.heappush(cola, (nuevo + h(vecino), contador, vecino))

    if meta not in g_score:
        return Resultado("A*", [], float("inf"), expandidos, len(padre))
    camino, actual = [], meta
    while actual is not None:
        camino.append(actual)
        actual = padre[actual]
    camino.reverse()
    return Resultado("A*", camino, g_score[meta], expandidos, len(padre))


def heuristica_euclidiana(g: nx.DiGraph, meta: Estado, atributo_pos: str = "pos"
                          ) -> Callable[[Estado], float]:
    """
    Construye h(n) = (menor coste por unidad de distancia) × distancia(n, meta).

    Como el factor es el MÍNIMO coste/longitud de todas las aristas del grafo,
    la heurística nunca sobreestima el coste real -> es ADMISIBLE (y consistente).
    """
    px = {n: g.nodes[n][atributo_pos] for n in g.nodes()}
    minimo = float("inf")
    for o, d, data in g.edges(data=True):
        largo = ((px[o][0] - px[d][0]) ** 2 + (px[o][1] - px[d][1]) ** 2) ** 0.5
        if largo > 0:
            minimo = min(minimo, data["weight"] / largo)

    def h(estado: Estado) -> float:
        if estado == meta:
            return 0.0
        dx = px[estado][0] - px[meta][0]
        dy = px[estado][1] - px[meta][1]
        return minimo * (dx * dx + dy * dy) ** 0.5
```

```
    return h
```

A.4 busqueda.py — verificación por fuerza bruta

```
def fuerza_bruta_tsp(tiempos: Dict[Tuple[str, str], float], base: str,
                     puntos: Sequence[str]) -> Tuple[List[str], float]:
    """
    Enumera todos los órdenes de entrega posibles y devuelve el mejor
    (problema del agente viajero con regreso a la base). Sirve como
    referencia independiente para validar la solución de Dijkstra.
    """
    mejor_camino: List[str] = []
    mejor_coste = float("inf")
    for orden in permutations(puntos):
        coste = tiempos[(base, orden[0])]
        for a, b in zip(orden, orden[1:]):
            coste += tiempos[(a, b)]
        coste += tiempos[(orden[-1], base)]
        if coste < mejor_coste:
            mejor_coste = coste
            mejor_camino = [base, *orden, base]
    return mejor_camino, mejor_coste
```